```
#include <stdint.h>

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

#include "driver/gpio.h"
#include "esp_err.h"
#include "esp_timer.h"

#include "soc/gpio_reg.h"
#include "soc/soc.h"

/*
 * ARDRA LAB Assignment: FW10779
 * LED Pattern Using External Interrupt
 *
 * Target: Original ESP32 / ESP32 DevKit V1
 * Framework: ESP-IDF 6.0.1
 *
 * Common-anode RGB LED:
 * LOW  = LED ON
 * HIGH = LED OFF
 */

#define LED_RED_GPIO GPIO_NUM_25
#define LED_GREEN_GPIO GPIO_NUM_26
```

```
#define LED_BLUE_GPIO GPIO_NUM_27
#define BUTTON_GPIO GPIO_NUM_33

#define LED_RED_MASK (1UL << LED_RED_GPIO)
#define LED_GREEN_MASK (1UL << LED_GREEN_GPIO)
#define LED_BLUE_MASK (1UL << LED_BLUE_GPIO)

#define LED_MASK (LED_RED_MASK | LED_GREEN_MASK | LED_BLUE_MASK)

#define DEBOUNCE_MS 30
#define PRESS_LOCKOUT_MS 150

static TaskHandle_t button_task_handle = NULL;
static uint8_t pattern_state = 0;

/*
 * Drive the RGB LED using ESP32 GPIO registers.
 *
 * State 0: All OFF
 * State 1: Red ON
 * State 2: Green ON
 * State 3: Blue ON
 * State 4: All ON
 */

static void update_led_pattern(uint8_t state){
```

```
uint32_t on_mask = 0;

switch(state){
 case 1:
  on_mask = LED_RED_MASK;
  break;
 case 2:
  on_mask = LED_GREEN_MASK;
  break;
 case 3:
  on_mask = LED_BLUE_MASK;
  break;
 case 4:
  on_mask = LED_MASK;
  break;
 default:
  on_mask = 0;
  break;
}

/*
  * Common-anode logic:
  * Set OFF channels HIGH.
  * Clear ON channels LOW.
  *
  * W1TS and W1TC registers modify only
```

```
    * the specified GPIO bits.
    */
  REG_WRITE(GPIO_OUT_W1TS_REG, LED_MASK & ~on_mask);
  REG_WRITE(GPIO_OUT_W1TC_REG, on_mask);
}

/*
 * GPIO interrupt service routine.
 * Keep the ISR short: notify the task only.
 */

static void IRAM_ATTR button_isr_handler(void *arg){
  BaseType_t higher_priority_task_woken = pdFALSE;

  vTaskNotifyGiveFromISR(
    button_task_handle,
    &higher_priority_task_woken
  );

  if(higher_priority_task_woken == pdTRUE){
    portYIELD_FROM_ISR();
  }
}

/*
 * Button task:
```

```
 * 1. Wait for interrupt notification.
 * 2. Allow contact bounce to settle.
 * 3. Drain accumulated notifications.
 * 4. Verify that the button is still pressed.
 * 5. Apply a lockout and advance the state.
 */
static void button_task(void *arg){
 int64_t last_valid_press_us = 0;

 while(1){
  /* Block until the GPIO ISR notifies this task. */
  ulTaskNotifyTake(pdTRUE, portMAX_DELAY);

  /* Debounce outside the ISR. */
  vTaskDelay(pdMS_TO_TICKS(DEBOUNCE_MS));

  /* Discard additional notifications from contact bounce.*/
  (void)ulTaskNotifyTake(pdTRUE, 0);

  /* Active-low button: LOW means pressed. */
  if(gpio_get_level(BUTTON_GPIO) == 0){
   int64_t now_us = esp_timer_get_time();

   if ((now_us - last_valid_press_us) >= ((int64_t)PRESS_LOCKOUT_MS * 1000)){
    last_valid_press_us = now_us;
```

```
      /*
       * 0 -> 1 -> 2 -> 3 -> 4 -> 1 ...
       */
      pattern_state++;

      if (pattern_state > 4){
        pattern_state = 1;
      }

      update_led_pattern(pattern_state);
    }
   }
  }
}

void app_main(void){
  /* Preload OFF levels before enabling LED outputs. */
  REG_WRITE(GPIO_OUT_W1TS_REG, LED_MASK);

  /* Configure RGB LED pins as outputs. */
  gpio_config_t led_config = {
    .pin_bit_mask = LED_MASK,
    .mode = GPIO_MODE_OUTPUT,
    .pull_up_en = GPIO_PULLUP_DISABLE,
    .pull_down_en = GPIO_PULLDOWN_DISABLE,
    .intr_type = GPIO_INTR_DISABLE
```

```
};

ESP_ERROR_CHECK(gpio_config(&led_config));

/* Configure the button as an input with internal pull-up. */
gpio_config_t button_config = {
  .pin_bit_mask = (1ULL << BUTTON_GPIO),
  .mode = GPIO_MODE_INPUT,
  .pull_up_en = GPIO_PULLUP_ENABLE,
  .pull_down_en = GPIO_PULLDOWN_DISABLE,
  .intr_type = GPIO_INTR_NEGEDGE
};

ESP_ERROR_CHECK(gpio_config(&button_config));

/* Establish the initial all-OFF state. */
pattern_state = 0;
update_led_pattern(pattern_state);

/* Create the task before enabling button interrupts. */
BaseType_t task_created = xTaskCreate(
  button_task,
  "button_task",
  3072,
  NULL,
  10,
```

```
    &button_task_handle
  );

  configASSERT(task_created == pdPASS);

  /* Install the GPIO interrupt service. */
  ESP_ERROR_CHECK(gpio_install_isr_service(0));

  /* Attach the ISR to the button GPIO. */
  ESP_ERROR_CHECK(
    gpio_isr_handler_add(
      BUTTON_GPIO,
      button_isr_handler,
      NULL
    )
  );
}
```